ALAB 351.1 — Introduction to Python and Computer Programming

Name: James Sloan

Date: August 13, 2026

Part 1: Written Exercise — Understanding Programming

1. What is computer programming, and why is it important in today's technology-driven world?

Computer programming is writing step-by-step instructions that tell a computer what to do. A computer cannot decide anything on its own, so a person has to spell out every step in a language the machine understands. It matters because almost everything we use daily — phones, banking, cars, hospitals — runs on software that someone had to write.

2. Briefly describe what Python is and one reason it is a popular programming language for beginners.

Python is a general-purpose programming language used for websites, data analysis, automation, and artificial intelligence. It is popular with beginners because the code reads a lot like plain English, so you spend more time learning how to solve the problem and less time fighting the syntax.

3. What do you need (software/tools) to start writing and running Python programs on your own computer?

You need the Python interpreter installed, which is the program that actually runs your code. You also want a text editor or IDE such as VS Code to write and save the files. Finally you need a terminal or command prompt to run the file with a command like `python3 hello.py`.

Part 2: Coding Exercise — Your First Python Program

Code

```
print("Hello, World!")
print("Welcome to Python programming.")
```

Output

```
Hello, World!
Welcome to Python programming.
```

Challenges Encountered and How I Resolved Them

The main challenge was getting the output to match the required text exactly. On my first pass I dropped the period at the end of the second line, and the two lines looked identical at a glance. I fixed it by copying the required text directly from the assignment instead of retyping it, then comparing my saved output against it line by line until they matched.

The other small issue was running the file. Typing `python` in my terminal did not point at the version I had installed, so the script would not start. Using `python3 hello.py` from the folder containing the file worked, and I confirmed it by seeing both lines print in order.

Part 3: Research Exercise — Interactive Greeting Script

Code

```
# Print the same two lines from Part 2.
print("Hello, World!")
print("Welcome to Python programming.")

# Ask the user for their name. input() shows the question and waits for typing.
name = input("What is your name? ")

# .strip() removes spaces at the start and end, so "   " becomes "".
name = name.strip()

# If the name is empty, use a friendly fallback instead of printing a blank name.
if name == "":
    name = "friend"

# An f-string lets me drop the variable straight into the sentence.
print(f"Hello, {name}! Glad to have you learning Python.")
```

Sample Runs

Every run prints the two Part 2 lines first, then the prompt. The table below shows what was typed at the `What is your name?` prompt and the greeting line that resulted.

Input	Output
`Sloan`	`Hello, Sloan! Glad to have you learning Python.`
`José`	`Hello, José! Glad to have you learning Python.`
`O'Brien-Ñ 王 🙂`	`Hello, O'Brien-Ñ 王 🙂! Glad to have you learning Python.`
(empty string — pressed Enter)	`Hello, friend! Glad to have you learning Python.`
(spaces only)	`Hello, friend! Glad to have you learning Python.`

Full transcript of one run:

```
Hello, World!
Welcome to Python programming.
What is your name? Sloan
Hello, Sloan! Glad to have you learning Python.
```

What I Researched

I had to look up three things. First, `input()` — how to pause the program, show a question, and store whatever the user types in a variable. Second, f-strings, which let me put a variable directly inside a string by writing `f"..."` and wrapping the variable in curly braces; that was easier to read than gluing strings together with `+`. Third, how to deal with an empty answer. Pressing Enter without typing gives back an empty string rather than an error, so nothing crashes — it just prints a greeting with a blank space where the name should be. I found `.strip()`, which removes leading and trailing spaces, so a name of only spaces also becomes empty. Then a simple `if` check swaps in `friend` when the name is empty, which covers both cases with one condition.

Unusual characters needed no extra work. Accents, apostrophes, hyphens, non-Latin characters, and emoji all printed correctly because Python 3 handles Unicode text by default.